Desktop-Fakturierungsanwendung

SCHLANKE, LOKAL BETRIEBENE FAKTURIERUNG FÜR KLEINSTUNTERNEHMEN

> *Abschlusspräsentation im Modul Software Engineering 1, TH Mannheim (07/2026) — historischer Stand der Version 1.0 (Swing, JSON, manuelle DI). Aktueller Stand 3.0: siehe Fallstudie.*

Agenda

1. **Einleitung** — Kontext & Ziel, Architektur
2. **Das Programm**
   - Komponente C — Kundenverwaltung
   - Komponente B — Produktverwaltung
   - Komponente A — Dokumentenzyklus
   - Komponente D — Programmoberfläche
3. **Das Entwicklungsprojekt**
   - Ergebnisse der Modultestpläne
   - Wo wurde KI eingesetzt – und wie gut?
   - Was würden wir nächstes Mal anders machen?

Projektorganisation

12 Studierende in vier Gruppen zu je drei Personen — je Gruppe eine fachliche Komponente:

KOMPONENTE	VERANTWORTUNG
A	Prozess / Dokumentenzyklus
B	Produktverwaltung
C	Kundenverwaltung
D	Programmoberfläche

Querschnitt (`gemeinsam`): EreignisBus, JSON-Persistenz, CSV-Hilfe – von allen Komponenten genutzt.

Kontext & Ziel

Problem: SaaS-Fakturierung kostet laufend Lizenzgebühren; Open-Source-Alternativen (z. B. *Fakturama*) sind im Betrieb anspruchsvoll.

Unsere Lösung: schlanke Desktop-Anwendung mit voller Datensouveränität – ohne Cloud.

ZIEL	INHALT
PZ-01	Digitale Verwaltung von Kunden- & Produktstammdaten (CRUD)
PZ-02	Vollständiger Dokumentenzyklus: Angebot → AB → Lieferschein → Rechnung
PZ-03	Bedienbar ohne Vorkenntnisse (Rechnung in < 10 Min)
PZ-04	Lokale Datenhaltung; Datenschutzbewertung gesondert

Nichtziele der Version 1.0: Mehrbenutzer/Netzwerk, vollständige Buchhaltung, E-Rechnung.

Architektur im Überblick

Vier Fachmodule + gemeinsame Infrastruktur, manuelle Dependency Injection in `Main.java` (kein Framework).

- **Repository-Interfaces** je Domäne mit JSON-Implementierungen; atomare Schreibvorgänge (`JsonPersistenz.schreibeAtomar`)
- **EreignisBus** (Observer): Services melden Datenänderungen, GUI-Panels aktualisieren sich selbst
- **Snapshot-Prinzip** in Belegen + **Löschsperren** (referenzielle Integrität)

Das Programm

VIER KOMPONENTEN

Komponente C — Kundenverwaltung

ZIEL *(LASTEN-/PFLICHTENHEFT)*

Kundenstammdaten anlegen, ändern, suchen, löschen *(BA-01–04)* · eindeutige Kundennummer · **keine Löschung bei verknüpften Dokumenten** *(GR-04)*

TECHNISCHE UMSETZUNG

- **Referenzielle Integrität** ohne Datenbank: Löschsperre über eine Referenzprüfung
- Automatische Nummernvergabe & Pflichtfeld-/E-Mail-Validierung

`KundenRepository` · `EinfacherKundennummernGenerator` · `KundenReferenzPruefung`

LIVE-DEMO

Kunde anlegen → automatische Nummer · suchen / sortieren · ändern · löschen

Komponente B — Produktverwaltung

ZIEL *(LASTEN-/PFLICHTENHEFT)*

Produkte anlegen, ändern, suchen, löschen *(BA-05–08)* · Nettopreis + Steuersatz · **nur gültige Eingaben**, Löschsperre

TECHNISCHE UMSETZUNG

- **Validierung** lehnt negative Preise & unzulässige Steuersätze ab; Produktnummer unveränderlich
- Gleiches Repository-/Generator-Muster wie Komponente C

  `ProduktRepository` · `EinfacherProduktnummernGenerator` · `ProduktReferenzPruefung`

LIVE-DEMO

Produkt anlegen → `P-000042` · neg. Preis & Steuersatz `0.15` **abgelehnt** · ändern · suchen · Löschsperre

Komponente A — Prozess / Dokumentenzyklus

ZIEL *(LASTEN-/PFLICHTENHEFT)*

Angebot → Auftragsbestätigung → Lieferschein → Rechnung *(BA-09–12)* · lückenlose Belegnummern *(GR-01)* · Steuer & Zahlungsziel · § 14 UStG · Storno *(BA-14)*

TECHNISCHE UMSETZUNG

- **Snapshot-Prinzip**: alte Rechnungen bleiben preisstabil, auch wenn sich Produktpreise ändern *(GR-03)*
- **Lückenlose** Belegnummern (Projektinvariante GR-01) · Folgebeleg übernimmt Daten + Rückreferenz *(GR-05)*

  `DokumentRepository` · `BelegnummernGenerator` · `PdfBoxPdfExporter`

LIVE-DEMO

Angebot erstellen → Folgebeleg zu Rechnung → **PDF-Export**

Komponente D — Programmoberfläche

ZIEL *(LASTEN-/PFLICHTENHEFT)*

Geführte Rechnungserstellung *(BA-13)* · Navigation, Pflichtfeldhinweise *(Q-09)*, Statusanzeige · bedienbar **ohne Vorkenntnisse** *(PZ-03)*

TECHNISCHE UMSETZUNG

- **EreignisBus (Observer)**: Panels aktualisieren sich automatisch nach Datenänderung
- Fachlogik strikt getrennt — GUI ruft nur die Services der Komponenten A–C

  Swing + FlatLaf · `HauptFenster` · `RechnungsWizardController`

LIVE-DEMO

Wizard: Kunde → Positionen → Bestätigen → Zusammenfassung → Speichern · Statusfilter „offen“ · Pflichtfeldhinweis · PDF-/CSV-Export

Das Entwicklungsprojekt

MODULTESTS · KI-EINSATZ · LESSONS LEARNED

Ergebnisse der Modultestpläne

71 / 71 TESTFÄLLE BESTANDEN — 100 %, 0 FEHLER

BEREICH	TESTFÄLLE	ERGEBNIS
A — Dokumentenzyklus	18	bestanden
B — Produktverwaltung	14	bestanden
C — Kundenverwaltung	14	bestanden
D — Programmoberfläche	15	bestanden
Infrastruktur	6	bestanden
Performance	4	Schranken eingehalten
Gesamt	**71**	**100 % bestanden**

Deterministische JUnit-5-Tests, Nachbarkomponenten als Stubs/Mocks. Traceability **Anforderung** → **Code** (`@DisplayName`) → **Testfall**; spezifiziert in `Modultestplan.md`, nachgewiesen in `Anforderungsabgleich.md`.

> *Historische Abnahme v1.0. Für 3.1.0 gelten zusätzliche technische Freigabeprüfungen; Usability-Studien sind weiterhin offen.*

Wo wurde KI eingesetzt – und wie gut?

Eingesetzt für

- **Dokumentation:** Entwürfe für Lasten-/Pflichtenhefte, Modultestplan, Traceability-Matrix
- **Code:** Boilerplate der Repository-/Service-Schicht, JSON-Persistenz, CSV-Export, JUnit-Testgerüste, Verwendung von Design-Patterns, Optimierungen
- **Diagramme & Tooling:** PlantUML-Quellen, Maven-Konfiguration, pandoc-/PDF-Workflow

Bewertung

- **Stärken:** Hohes Tempo bei Produktion, Tests & Doku; konsistente deutsche Fachsprache; schnelle Iteration nach Feedback
- **Schwächen:** Vorschläge teils zu generisch

Was würden wir nächstes Mal anders machen?

- **Früher integrieren:** modulübergreifende Tests nicht erst spät im V-Modell
- **Schnittstellen-Verträge** zwischen den Komponenten A–D verbindlich zu Projektbeginn festlegen
- **Kleinere, häufigere Commits** + einheitliche Anforderungen an Commits
- **KI-Output gezielter prüfen** statt übernehmen – besonders bei Fachlogik
- Aufgaben-/Lastverteilung über die Git-History früh sichtbar machen

Vielen Dank!

Version 3.1.0

- Historische Beträge und Einheiten als unveränderliche Snapshots
- EN 16931 CII mit Schema-/Schematron-Prüfung; keine XRechnung
- Atomare Sicherungen und Upgradeprobe auf restaurierter Kopie
- Start mit 5.000 Kunden/Produkten in 3,5–3,7 s (Vorgabe ≤ 5 s)
- Release prüft Plattformen, MSI, Upgrade mit Daten, Sicherheit und Leistung

Stand: 18.10.2026. Ergebnisse im Modultestbericht.

Portfolio: aktueller Schwerpunkt

- Windows-Demo: ein Kunde, zwei Produkte, vollständige Belegkette
- Neue Rechnung: 185,00 EUR netto + 35,15 EUR Steuer = 220,15 EUR
- Produktpreis ändern: bestehende Rechnung bleibt unverändert
- PDF, validiertes CII-XML, vollständiger CSV-Export und Sicherung

Vorbereitung und Drei-Minuten-Drehbuch: Portfolio-Abschluss.md

Nachweise und Grenzen offen benennen

- Usability mit fünf Personen: derzeit zurückgestellt, nicht nachgewiesen
- Startzeit: zwei von drei letzten Läufen über fünf Sekunden
- Security-Scan: NVD HTTP 429; MSI und weitere Plattformläufe offen
- Mahnwesen, zusätzliche Steuerszenarien und native Pakete: später

Portfolio-Vorführung ist keine Produktiv- oder RC-Freigabe.

Eigenleistung und KI-Unterstützung

- Ursprung: SE1-Teamprojekt; anschließende individuelle Weiterentwicklung
- Claude: Unterstützung bei Code, Dokumentation und Umbauten
- Codex: Review, Remediation, Regressionen und Portfolio-Aufbereitung
- Eigene Verantwortung: übernommene Regeln verstehen und erläutern

Gesprächsbeispiele: Transaktionen, historische Beträge, sichere Migration.